OptiTrace

Network API execution optimisation for MTN

MTN Group Technology Hackathon 2026
Concept proposal | 20 October 2026

Executive recommendation

Develop OptiTrace as a performance layer above MTN Chenosis and existing policy engines. The proposed runtime would choose efficient ways to execute approved enterprise decision rules using MTN and permitted partner APIs. Its commercial purpose is to reduce delivery cost and delay while preserving the required decision behaviour.

Begin with one account verification policy, three network capabilities and one market. The team should demonstrate the same workload through competent existing implementations and OptiTrace, then compare required decisions, compulsory checks, cost and response time. Performance improvement is a hypothesis to test; this proposal reports no measured savings.

Customer and value proposition

The initial platform customer would be MTN Chenosis or the relevant enterprise product owner. Banks, fintechs and software providers consuming a managed verification service would be the downstream users. OptiTrace would help MTN serve those businesses more efficiently, subject to an appropriate commercial model.

Why this approach fits

MTN organises its strategy around Connectivity, Fintech and Digital Infrastructure. Chenosis already supplies MTN and third-party APIs, making it a credible platform above which to test an execution improvement.

The Databricks comparison concerns the product strategy: build a reliable commercial layer around existing technology and earn adoption through useful performance improvements. Databricks was founded by the creators of Apache Spark, which remains central to its platform. OptiTrace would need to establish its own measurable advantage.

Decision required

Proceed to a bounded prototype and confirm the workload with Chenosis. Continue only if OptiTrace beats a strong baseline on execution economics while meeting the agreed

correctness and latency requirements. No live customer data or production access is required for the first demonstration.

The business problem and proposed solution

Enterprise applications can combine several network checks before completing an account action. These may include Number Verification, SIM Swap and Device Swap. Each dependency can introduce processing expense, partner charges, delay or an availability problem. Actual costs and supported capabilities must be confirmed for the chosen market.

The opportunity is to examine whether each request retrieves more information, repeats more work or waits longer than its approved policy requires. We should measure this against a well-designed implementation rather than assume existing MTN workflows are inefficient.

How OptiTrace would work

Stage	Proposed behaviour
Read the contract	Load the approved policy, required outputs, compulsory checks, permissions, freshness limits and latency requirement.
Plan the evaluation	Identify valid execution paths and estimate their cost, response time and failure behaviour.
Execute the plan	Call the permitted services in sequence or in parallel. Reuse information only when its scope and freshness allow it.
Stop or continue	Stop only when the required outputs are established and all compulsory work is complete. Otherwise fetch more information or use the approved fallback.
Record the evidence	Return the result with policy version, checks performed, source timestamps, execution trace and metered cost.

Illustrative account verification example

Assume an approved policy requires additional verification whenever a recent SIM change is found. If that finding establishes the required action, further optional checks may be unnecessary. OptiTrace could stop after completing any remaining compulsory checks. A recent SIM change would trigger the agreed action; it would not establish that the customer is fraudulent.

This is an illustrative policy, not a description of MTN or a bank's current controls. If the contract requires a full numerical score, all contributing inputs, or a complete explanation,

OptiTrace must obtain enough information to deliver those outputs. It cannot claim equivalent service by returning less information.

Technical design and operating boundaries

Use an established policy engine as the foundation. Separate the approved decision logic from execution planning so that a performance model cannot rewrite the customer's rules. Open Policy Agent already supports optimisation and early exit; these features form part of the baseline to improve upon.

Component	Responsibility
Policy contract	Defines decisions, required evidence, compulsory checks, error handling and the supported policy subset.
Capability adapters	Translate the supported MTN or CAMARA interfaces into typed inputs while preserving timestamps, permissions and unavailable states.
Execution planner	Chooses among valid plans using service cost, observed latency, availability and expected ability to settle the decision.
Policy evaluator	Applies approved rules and checks whether stopping is valid. Keeps missing or stale information distinct from a reassuring result.
Trace and replay store	Retains synthetic or appropriately protected execution records for debugging, comparison and cost accounting.
Benchmark interface	Displays baseline and optimised results on the same workload and lets judges inspect individual traces.

The learning component

A small statistical model could estimate response times, availability and how often a particular result settles a policy. The planner would use these estimates to rank valid execution plans. A rules-based planner should be built first; a learned planner is justified only if it improves the benchmark.

Conditions for preserving behaviour

- Compare policies on the same recorded source observations or a defined observation window. Live signals may change between requests, so reordering requires explicit freshness semantics.
- Treat policy errors, timeouts and unavailable data according to the approved contract. An absent signal must never silently become a low-risk result.
- Do not skip legally or contractually compulsory work. Reuse data only within its authorised tenant, purpose, consent and freshness boundaries.

- Support a small, explicitly defined class of policies first. Opaque models requiring every input may offer no valid opportunity to stop early.

MTN business value and commercial model

OptiTrace should be assessed through contribution per completed service, as well as customer experience. A faster or cheaper workflow is commercially useful only if MTN can retain the benefit after pricing, partner charges, runtime costs and operational overhead.

Potential commercial mechanisms

- Managed evaluations: MTN could package several capabilities into an enterprise service with transparent pricing per completed evaluation.
- Enterprise runtime subscription: customers could pay for planning, policy execution, audit and operational support, with underlying API usage priced separately.
- Improved adoption: lower integration effort or better performance might help customers move into production and use the service more often. This requires pilot evidence and must not be counted as secured revenue.

GSMA identifies commercial packaging and embedding network APIs in broader enterprise capabilities as important to monetisation. That supports investigating the model, rather than establishing that a particular price or contract will work.

Hypothetical contribution example

All amounts below are invented assumptions in rand per completed managed evaluation. They are not MTN tariffs, observed costs or forecasts.

Item	Existing execution	Optimised execution
Customer price	R1.00	R1.00
Underlying service costs	R0.55	R0.30
Runtime cost	R0.05	R0.10
Contribution before other costs	R0.40	R0.60

Under these assumptions, the improvement is R0.20 per evaluation, or R200,000 across one million evaluations at unchanged volume. Fixed development, integration, support, compliance, taxes and other costs are excluded. The illustration is a contribution calculation, not net profit or return on investment.

Revenue cannibalisation must be tested

If MTN charges only for individual API calls, reducing calls could reduce revenue. The actual business case must compare current and proposed contracts, wholesale costs, minimum commitments and billing rules. Customer savings cannot automatically be recorded as MTN profit, and skipped calls must not be billed as calls performed.

Market evidence and defensibility

Chenosis provides an existing distribution and integration platform for MTN and third-party APIs. This establishes a relevant place to deploy an improvement; public product descriptions do not establish that its current execution is inefficient.

GSMA publishes a request from COURE Software and Systems for network APIs to strengthen its All Number Query fraud-prevention platform across African markets. This identifies a real organisation seeking the underlying capabilities. It does not establish that COURE wants OptiTrace, has agreed to integrate, or will pay for execution optimisation.

Existing capabilities to respect

Existing category	Relevant capability	Implication for OptiTrace
Chenosis	MTN and partner API access and enterprise integration.	Confirm current runtime features and internal plans before claiming a product gap.
Open Policy Agent	Policy evaluation, optimisation and early exit.	Benchmark against an appropriately configured policy engine. [5]
Trulioo	Intelligent verification workflows and cost optimisation.	The broad orchestration category already exists. [6]
Staged enterprise workflows	Developers can conditionally sequence checks.	A strong hand-designed cascade is a required comparator.

Where an advantage could develop

The proposed specialisation is efficient execution across MTN network capabilities with explicit market availability, data freshness, permission boundaries and failure semantics. The advantage must be demonstrated through lower delivery cost or delay while preserving the supported contract's required behaviour.

Reliable connectors, repeated deployment experience and operational performance data could make the product harder to reproduce over time. Tenant data must remain isolated

unless an approved legal and contractual basis permits a broader use. The prototype code and the general idea of orchestration are not durable advantages on their own.

Claims that require evidence

Do not claim a world-first platform, proven MTN savings, better fraud detection or production readiness. Confirm the remaining capability gap with Chenosis and establish the improvement through the benchmark before presenting a commercial advantage.

Benchmark design and acceptance criteria

Use the same labelled synthetic requests, recorded API observations and fault scenarios for every comparator. Record both the strongest baseline and any simpler baselines; the strongest baseline determines whether the project passes.

Required comparisons

- A full-input evaluator that retrieves all relevant inputs, including parallel retrieval where valid.
- A competent staged workflow with conditional checks and normal operational safeguards.
- An established policy engine configured with its available optimisation features.
- OptiTrace with rules-based planning, followed by a learned planner only if core behaviour is correct.

Measure	Proposed acceptance criterion
Required decision behaviour	No unexplained differences from the reference on held-out replay; equivalence tests for the supported policy subset.
Compulsory work	Every required check completes or follows the approved failure path. No savings from bypassing obligations.
Variable execution cost	Initial target: at least 20% lower than the strongest staged baseline on the agreed representative mix.
Latency	Meet the agreed p95 latency requirement and an agreed regression tolerance relative to the strong baseline.
Failure behaviour	Exercise missing signals, stale observations, rate limits, timeouts, permission errors and source failures.
Commercial outcome	Show contribution under actual billing assumptions; separate supplier costs, runtime cost and customer price.

These are proposed targets, not measured results. The latency requirement and representative workload must be agreed with the product owner before the final benchmark.

Evaluation discipline

Keep tuning requests separate from held-out requests. Measure warm and cold execution, service latency distributions and instrumentation overhead. Test difficult cases where all inputs are necessary, not only cases that favour early stopping. Include the planner's own cost and avoid double-counting a shared API result.

Decision agreement demonstrates compatibility with a reference policy, not that the policy correctly detects fraud. Synthetic results cannot establish production savings or real-world fraud outcomes. Finite tests also do not prove equivalence for arbitrary policies; a production claim needs stronger verification and a bounded supported contract.

Prototype delivery and validation plan

Minimum viable scope

Build one account verification policy using Number Verification, SIM Swap and Device Swap as faithful mocks or approved sandbox services. Support one market and one clear decision output, with compulsory checks and explicit fallback behaviour. Confirm the exact interface contracts before implementation.

- Create a small policy editor or configuration file and version each policy.
- Implement adapters, a reference evaluator, a staged baseline and a rules-based planner.
- Generate reproducible synthetic requests and API responses with configurable prices, latency and faults.
- Produce decision traces and a side-by-side benchmark showing decisions, checks, elapsed time and total metered cost.

Suggested demonstration

First, run an ordinary request through all implementations. Next, show a request where an optional check cannot change the required decision. Then show a case requiring every check. Introduce a timeout or stale signal and inspect the approved fallback. Finish with aggregate results against the strongest baseline, including requests where OptiTrace offers no benefit.

Evidence needed before a pilot

Validation owner	Evidence required
Chenosis product owner	A current policy or trace, the existing runtime baseline, confirmation of overlap and a real performance or cost constraint.
Enterprise integration partner	An identified account verification workflow, required outputs and willingness to evaluate a controlled integration.
MTN commercial lead	Actual tariffs, supplier costs, billing constraints and ownership of any savings.
Risk and technical reviewers	An approved policy contract, permissions, freshness limits, error behaviour and controlled rollout criteria.

Decision after the benchmark

Continue if the agreed workload shows meaningful net benefit without weakening required behaviour. Narrow the use case if savings appear only in specific policy families. Stop or redesign if a strong existing implementation achieves similar results, planner overhead removes the benefit, all inputs are routinely necessary, or the commercial model lowers MTN contribution.

The pitch should present OptiTrace as an execution improvement above existing MTN services, supported by a reproducible comparison and clearly stated limits. Production deployment would follow an authorised shadow evaluation, a bounded pilot and staged expansion.